

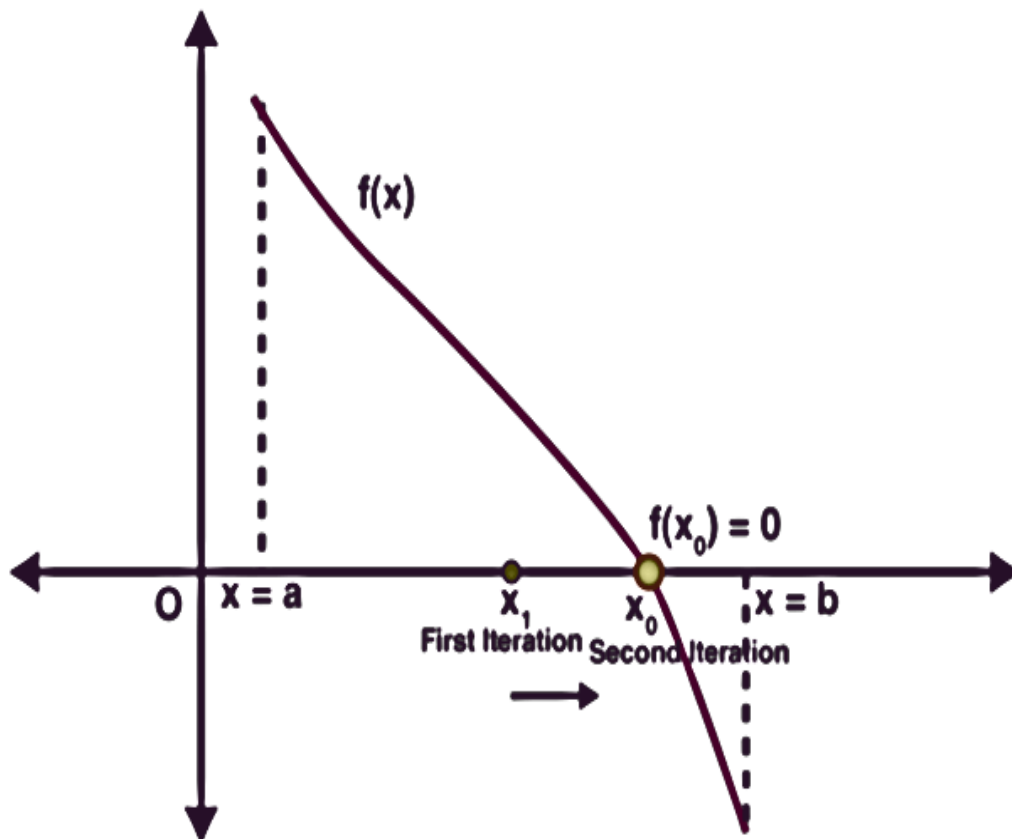
Experiment Name: Bisection Method in Python

1. Objectives:

- To understand the concept of the Bisection Method for finding roots of equations.
- To implement the Bisection Method using Python programming.
- To learn how this method gradually finds the root step by step within the set tolerance.

2. Introduction:

The Bisection Method is a simple numerical technique used to find the root of a nonlinear equation. The idea is to take an interval where the function changes sign, then repeatedly cut that interval in half and choose the part where the root lies. This process is based on the Intermediate Value Theorem, which says that if a continuous function changes sign between two points a and b , then there must be at least one root in that interval. The method is easy to apply, always converges if the interval is chosen properly, and gives reliable results.



3. Procedure

Algorithm: Bisection Method in Python

1. Start.
2. Define the function $f(x)$.
3. Choose two initial points **a** and **b** such that $f(a) \times f(b) < 0$ (the function changes sign).
4. Find the midpoint: $c = \frac{a+b}{2}$.
5. Calculate $f(c)$.
6. If $f(c) = 0$, then **c** is the root.
7. If $f(a) \times f(c) < 0$, then set $b = c$.
8. Otherwise, set $a = c$.
9. Repeat steps 4–5 until the interval $(b-a)$ becomes smaller than the given tolerance.
10. The midpoint of the final interval is taken as the approximate root.
11. Stop.

4. Required Hardware and Software:

- **Hardware:** AMD RYZEN 7, RAM 8 GB
- **Software:** PyCharm Community Edition 20251.1.1

5. Experiment Implementation:

Code:

```

def f(x):
    return x ** 3 + x ** 2 + x + 7
a = -3
b = -2
tol = 0.00005

def bisection(a, b, tol):
    if (f(a) * f(b) >= 0):
        print("Invalid")
        return None

print(f"{'Iteration':<12}{'a':<12}{'b':<12}{'f(a)':<12}{'f(b)':<12}{'c':<12}{'f(c)':<12}")
print("-" * 82)
iter_count = 1
while (b - a) > tol:
    c = (a + b) / 2
    fa = f(a)
    fb = f(b)
    fc = f(c)

print(f"{iter_count:<12}{a:<12.4f}{b:<12.4f}{fa:<12.4f}{fb:<12.4f}{c:<12.4f}{fc:<12.4f}")
    if fc == 0:
        return c
    elif (fa * fc < 0):
        b = c
    else:
        a = c
    iter_count += 1
return (a + b) / 2
root = bisection(a, b, tol)
if root is not None:
    print(f"\nRoot is: {root:.6f}")

```

Output:

Iteration	a	b	f(a)	f(b)	c	f(c)

1	-3.0000	-2.0000	-14.0000	1.0000	-2.5000	-4.8750
2	-2.5000	-2.0000	-4.8750	1.0000	-2.2500	-1.5781
3	-2.2500	-2.0000	-1.5781	1.0000	-2.1250	-0.2051
4	-2.1250	-2.0000	-0.2051	1.0000	-2.0625	0.4177
5	-2.1250	-2.0625	-0.2051	0.4177	-2.0938	0.1115
6	-2.1250	-2.0938	-0.2051	0.1115	-2.1094	-0.0455
7	-2.1094	-2.0938	-0.0455	0.1115	-2.1016	0.0333
8	-2.1094	-2.1016	-0.0455	0.0333	-2.1055	-0.0060
9	-2.1055	-2.1016	-0.0060	0.0333	-2.1035	0.0137
10	-2.1055	-2.1035	-0.0060	0.0137	-2.1045	0.0038
11	-2.1055	-2.1045	-0.0060	0.0038	-2.1050	-0.0011
12	-2.1050	-2.1045	-0.0011	0.0038	-2.1047	0.0014
13	-2.1050	-2.1047	-0.0011	0.0014	-2.1049	0.0001
14	-2.1050	-2.1049	-0.0011	0.0001	-2.1049	-0.0005
15	-2.1049	-2.1049	-0.0005	0.0001	-2.1049	-0.0002
Root is: -2.104874						

5. Discussion & Conclusion:

In this experiment, the Bisection Method was applied to approximate the root of a nonlinear equation. By repeatedly halving the interval and checking the sign of the function at the midpoint, the method steadily narrowed down the location of the root. In this process it is showed how a simple, iterative approach can reliably find roots, even when the function is nonlinear. Although it required multiple iterations, the method guaranteed convergence as long as the initial interval was chosen correctly. This shows the usefulness of Bisection method for solving equations where exact analytical solutions may be difficult.